

Le pattern Décorateur

Ajouter des options à un objet sans toucher à sa classe

Yassin, Robin, Émile

Qui parle, et de quoi ?

Yassin

Introduction aux patterns, le besoin, les mauvaises pistes et la solution

Transparents 3 à 12

Robin

Le pattern en détail : intention, diagrammes, participants, SOLID, limites

Transparents 13 à 22

Émile

Deux autres contextes, live coding et QCM

Transparents 23 à 35

Sommaire

1. Les design patterns en bref
2. Un besoin : la borne de commande du snack
3. Le pattern Décorateur
4. Décorateur et principes SOLID
5. Les limites du pattern
6. Deux autres contextes : un jeu vidéo et java.io
7. Live coding, QCM et sources

Un design pattern, c'est quoi ?

Une solution qui a fait ses preuves pour un problème de conception qui revient souvent.

Ce n'est pas du code à copier coller : c'est une façon d'organiser des classes et des objets, qu'on adapte à son propre contexte. Et c'est surtout un vocabulaire commun : dire « on met un décorateur » suffit pour que toute l'équipe comprenne.

1977

**Christopher Alexander, A
Pattern Language**

Un architecte décrit des solutions récurrentes pour construire maisons et villes.

1987

**Kent Beck et Ward
Cunningham**

Ils reprennent l'idée pour la programmation objet (conférence OOPSLA).

1994

Le « Gang of Four »

Gamma, Helm, Johnson et Vlissides publient Design Patterns : 23 patterns.

Les 23 patterns du GoF en trois familles

5 Création

Comment créer les objets sans que le code dépende des classes concrètes.

Ex. : Builder, Singleton, Fabrique abstraite

7 Structure

Comment assembler classes et objets pour former des structures plus grandes.

Ex. : Adaptateur, Composite, Décorateur

11 Comportement

Comment les objets communiquent et se répartissent le travail.

Ex. : Stratégie, Observateur, État

Chaque pattern est décrit avec quatre éléments : **un nom, un problème, une solution et des conséquences.**

Le Décorateur est un pattern de structure.

La borne de commande du snack

Le snack en face de l'IUT veut une borne de commande. Le client choisit un tacos, ajoute ses suppléments, et la borne imprime le ticket.

- Bases : tacos poulet (7,50 €) ou tacos végété (7,00 €)
- Suppléments : cheddar (+1,00 €), bacon (+1,50 €), gratiné (+0,80 €)
- On peut cumuler les suppléments, et même prendre deux fois le même
- Le ticket affiche la description complète et le prix total
- Le gérant ajoute un nouveau supplément presque tous les mois

SNACK DE L'IUT

Ticket n° 042

Tacos poulet

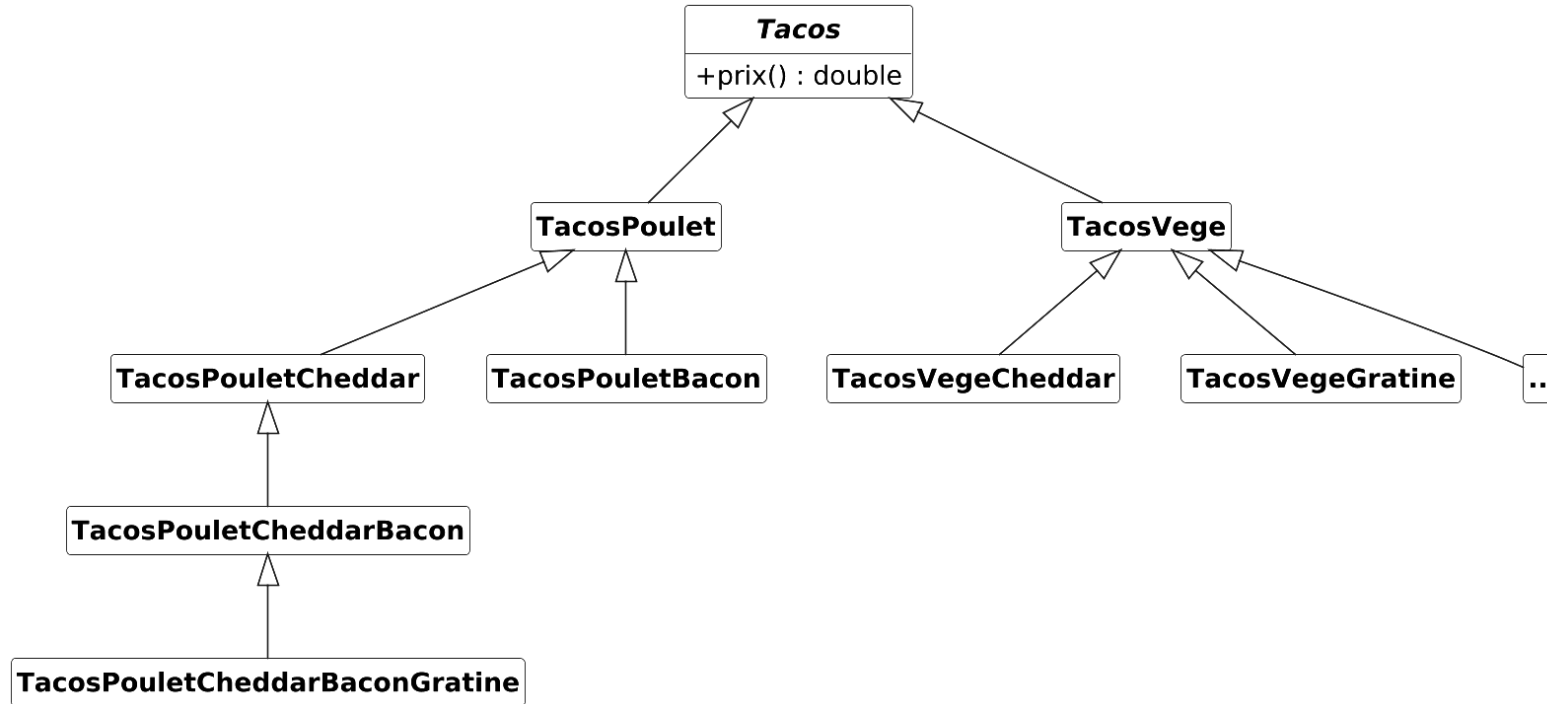
+ cheddar

+ cheddar

+ gratiné

TOTAL 10,30 EUR

Idée n° 1 : une classe par combinaison

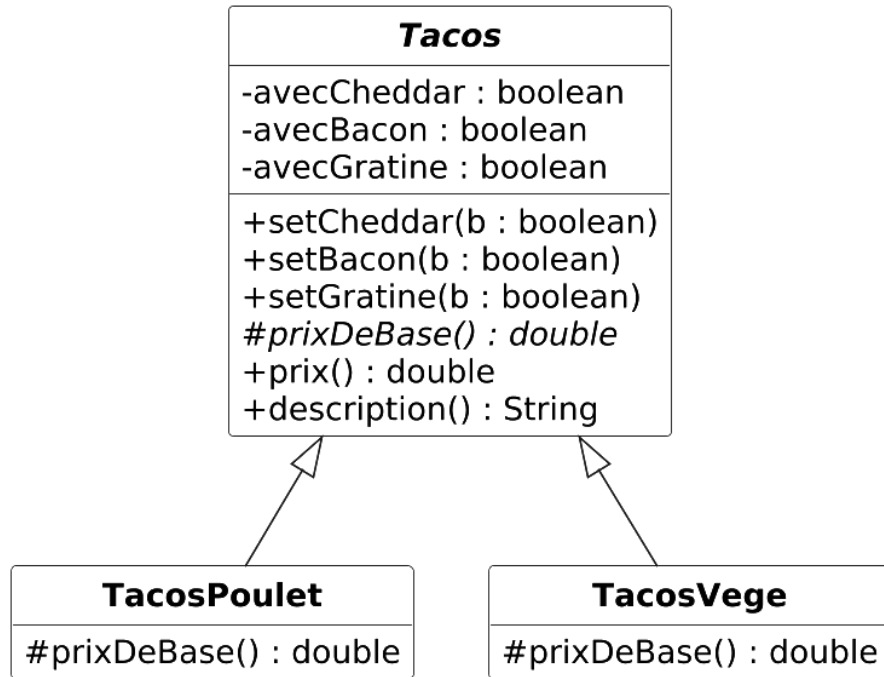


16

classes rien que pour 2 bases et 3 suppléments (2×2^3).

Un supplément de plus et le nombre de classes double. Et le double cheddar ? Il faut encore d'autres classes.

Idée n° 2 : des booléens dans Tacos



```

public abstract class Tacos {
    private boolean avecCheddar;
    private boolean avecBacon;
    private boolean avecGratine;

    protected abstract double prixDeBase();

    public double prix() {
        double total = prixDeBase();
        if (avecCheddar) total += 1.00;
        if (avecBacon)    total += 1.50;
        if (avecGratine) total += 0.80;
        return total;
    }
    // + description(), setters...
}

```

Pourquoi ces deux solutions ne tiennent pas

Modifier sans arrêt

Chaque nouveau supplément oblige à rouvrir Tacos : un attribut, un setter, un if dans prix() et dans description(). Puis tout retester.

Pas de double cheddar

Un booléen, c'est oui ou non. Impossible de compter deux fois le même supplément.

Des options absurdes

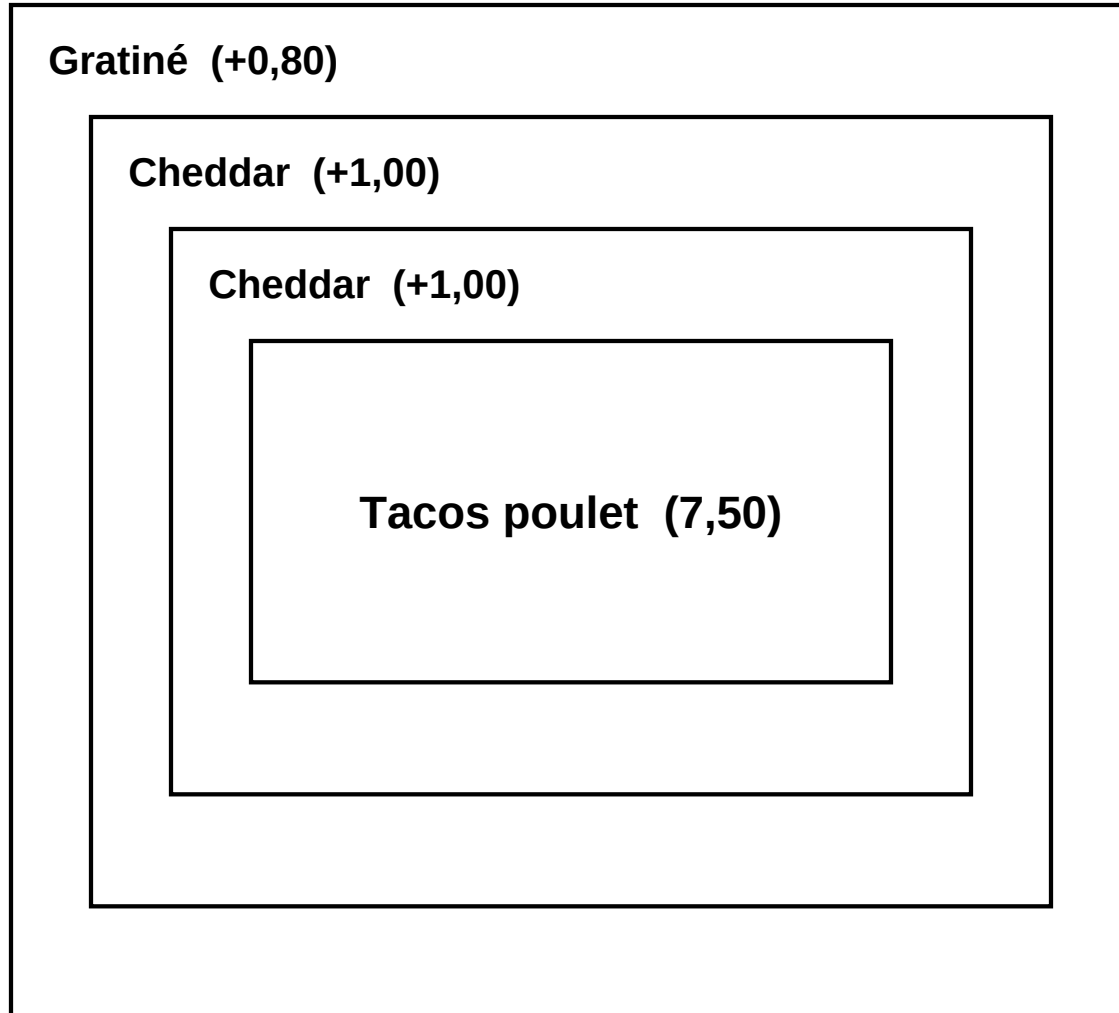
Un tacos végétarien hérite de avecBacon. Rien n'empêche de le passer à true.

Une classe fourre-tout

Tacos mélange la recette de base, la liste des options et le calcul du prix.

L'héritage fige les combinaisons à la compilation. Nous, on veut composer la commande au moment où le client la passe.

L'idée : emballer le tacos

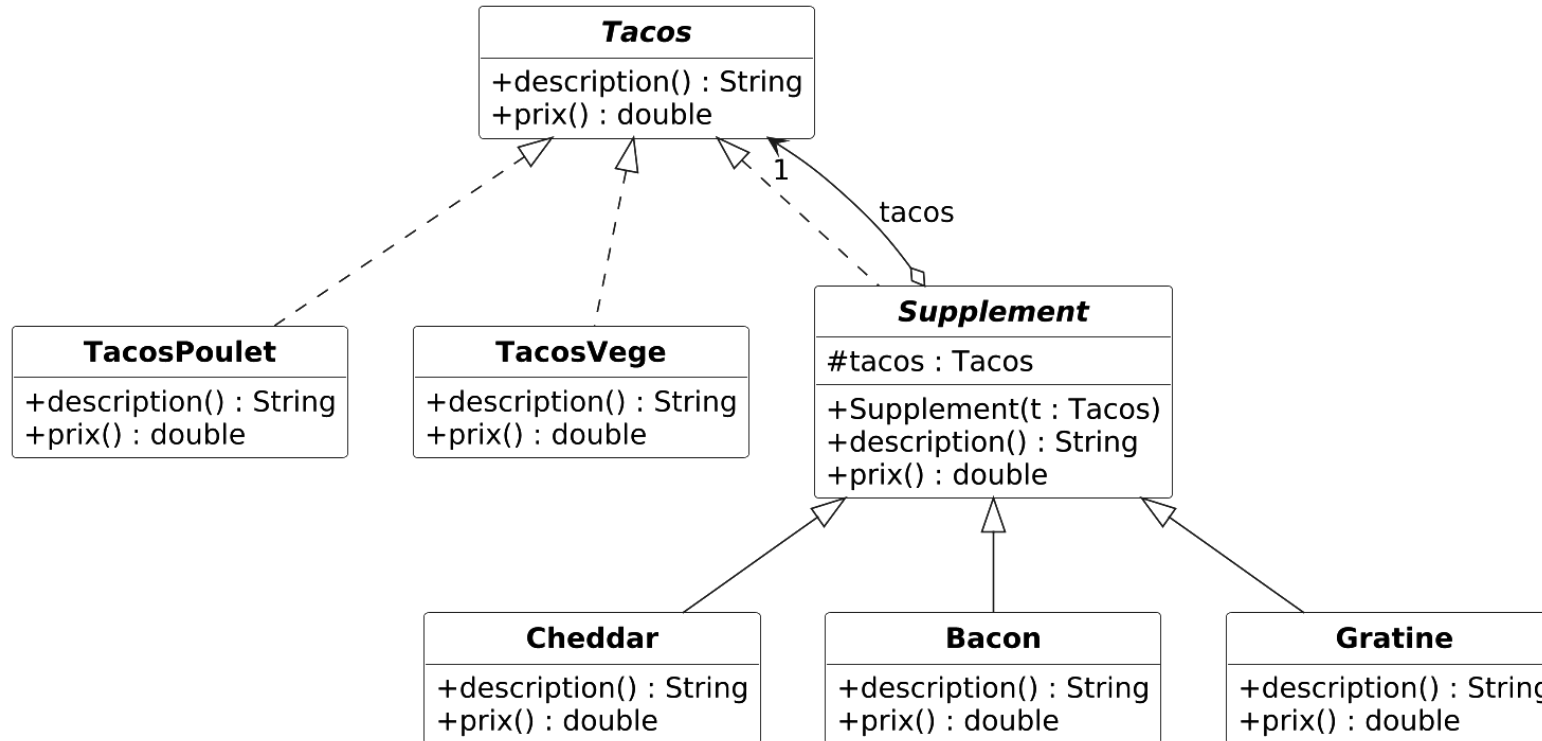


Chaque supplément devient un objet qui contient un tacos et qui se fait passer pour un tacos.

Quand on demande son prix au gratiné, il demande d'abord le prix de ce qu'il enveloppe, puis ajoute le sien. La demande descend jusqu'au tacos de base, et chaque couche ajoute sa part en remontant.

7,50 → 8,50 → 9,50 → 10,30 €

Les suppléments deviennent des objets



- Supplement implémente Tacos : pour la borne, un tacos décoré reste un tacos.
- Supplement contient aussi un Tacos : celui qu'il enveloppe.
- Un nouveau supplément = une nouvelle petite classe.
- Double cheddar : on emballe deux fois.

Le code

```

public interface Tacos {
    String description();
    double prix();
}

public abstract class Supplement implements Tacos {
    protected final Tacos tacos;
    protected Supplement(Tacos tacos) {
        this.tacos = tacos;
    }
}

public class Cheddar extends Supplement {
    public Cheddar(Tacos tacos) { super(tacos); }
    @Override public String description() {
        return tacos.description() + " + cheddar";
    }
    @Override public double prix() {
        return tacos.prix() + 1.00;
    }
}

```

```

public class TacosPoulet implements Tacos {
    public String description() {
        return "Tacos poulet";
    }
    public double prix() { return 7.50; }
}

// Côté borne de commande
Tacos commande = new TacosPoulet();
commande = new Cheddar(commande);
commande = new Cheddar(commande);
commande = new Gratine(commande);

System.out.println(commande.description()
    + " : " + commande.prix());
// Tacos poulet + cheddar + cheddar
//   + gratiné : 10.3

```

Du tacos au diagramme générique

Notre solution (snack)

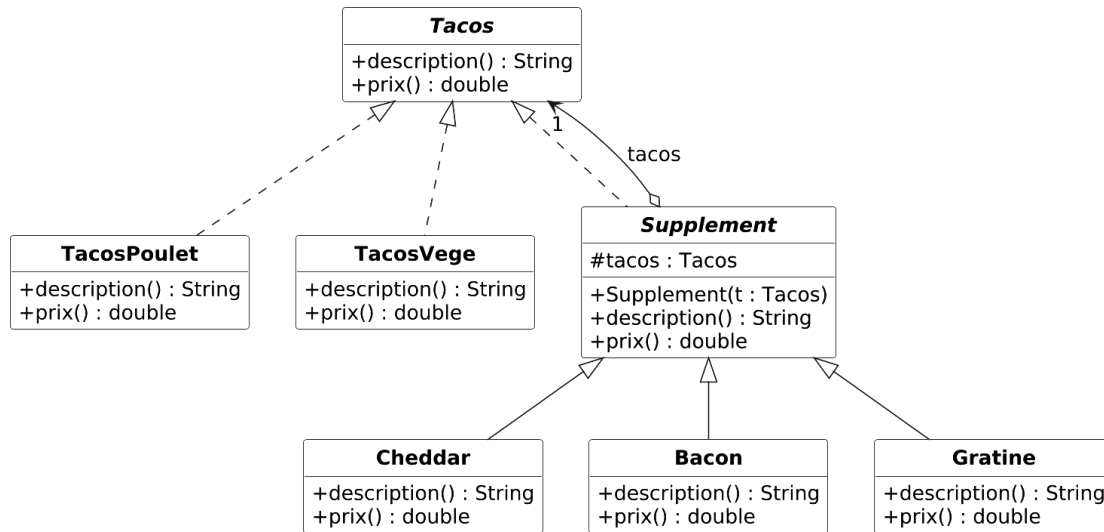
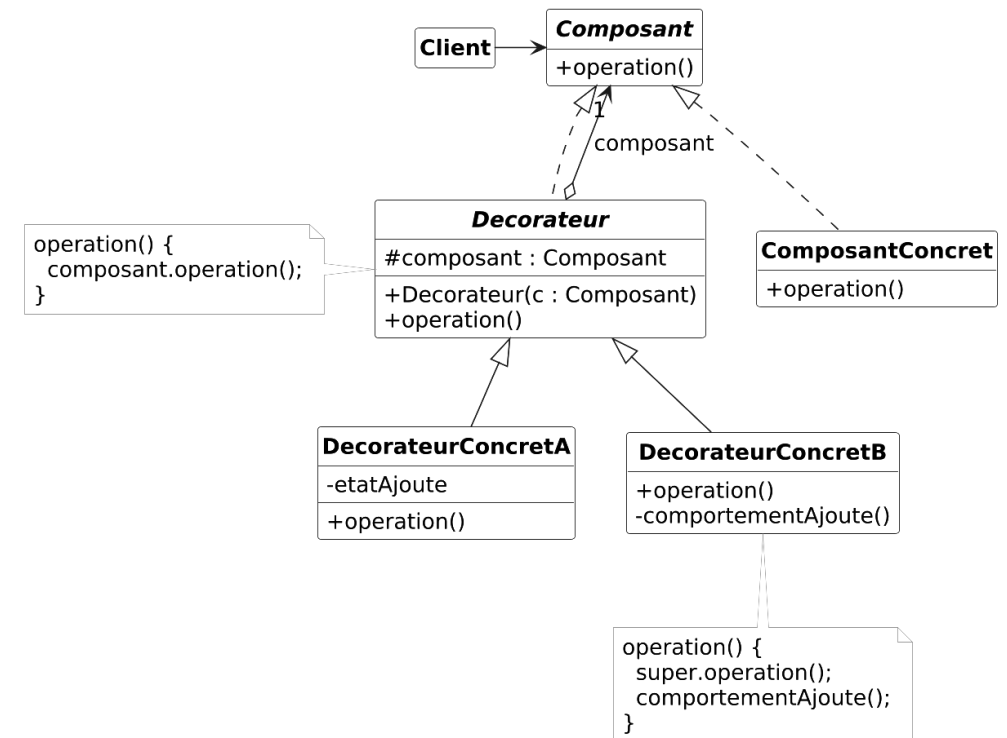


Diagramme générique



Tacos = Composant

TacosPoulet, TacosVege =
ComposantConcret

Supplement = Decorateur

Cheddar, Bacon, Gratine =
DecorateurConcret

Ce pattern s'appelle...

Décorateur

En anglais : Decorator. On le trouve aussi sous le nom de Wrapper (emballage), ce qui résume bien l'idée.

Type

Pattern de structure du GoF

Portée

Objet : l'assemblage se fait entre objets, pendant l'exécution, et pas entre classes à la compilation.

Problématique et intention

PROBLÉMATIQUE

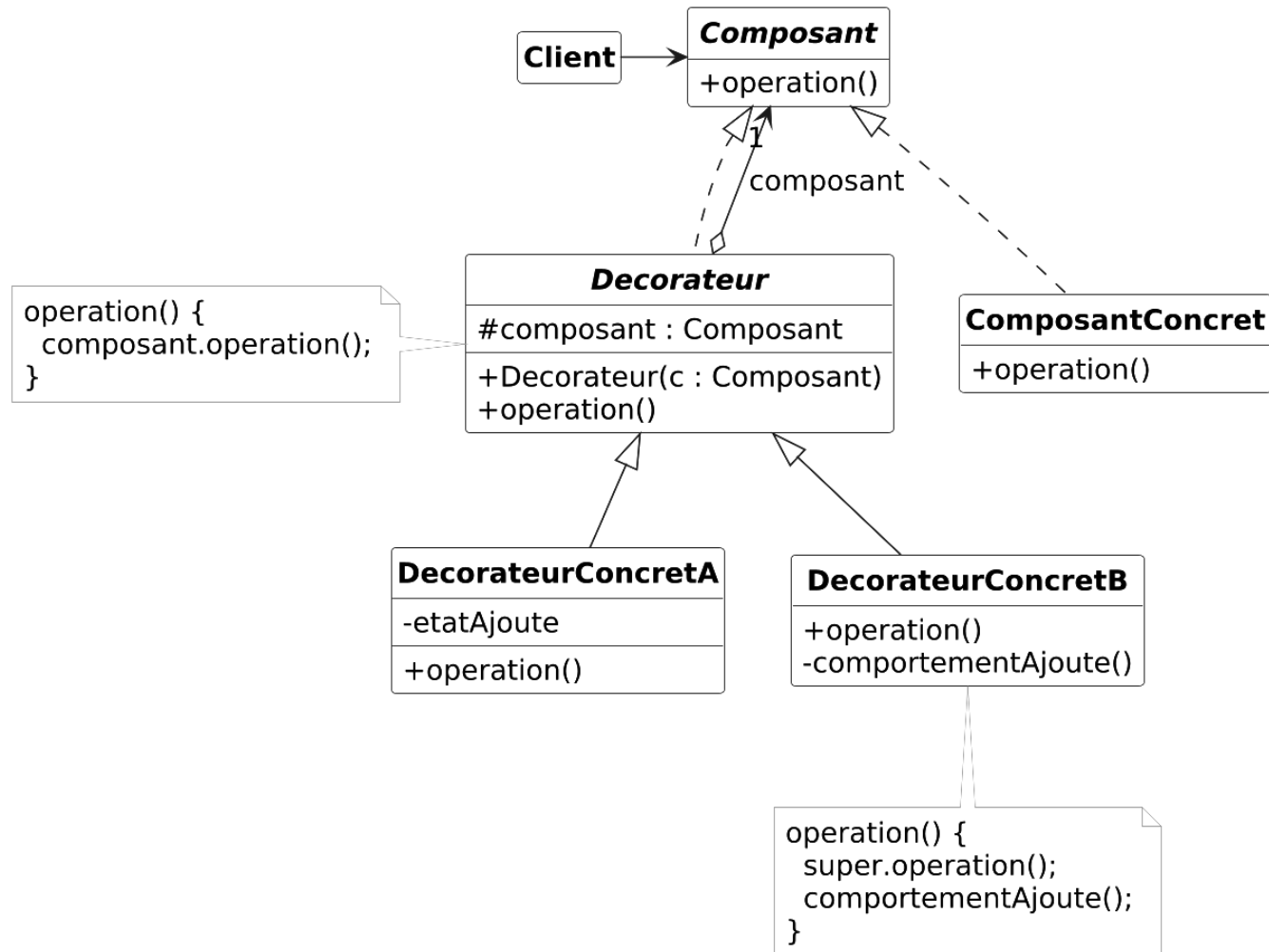
Comment ajouter des responsabilités à un objet, une par une et dans n'importe quel ordre, sans modifier sa classe et sans créer une sous-classe pour chaque combinaison ?

INTENTION

Ajouter dynamiquement de nouvelles responsabilités à un objet en l'enveloppant dans un autre objet qui a la même interface. C'est une alternative plus souple que l'héritage pour étendre un comportement.

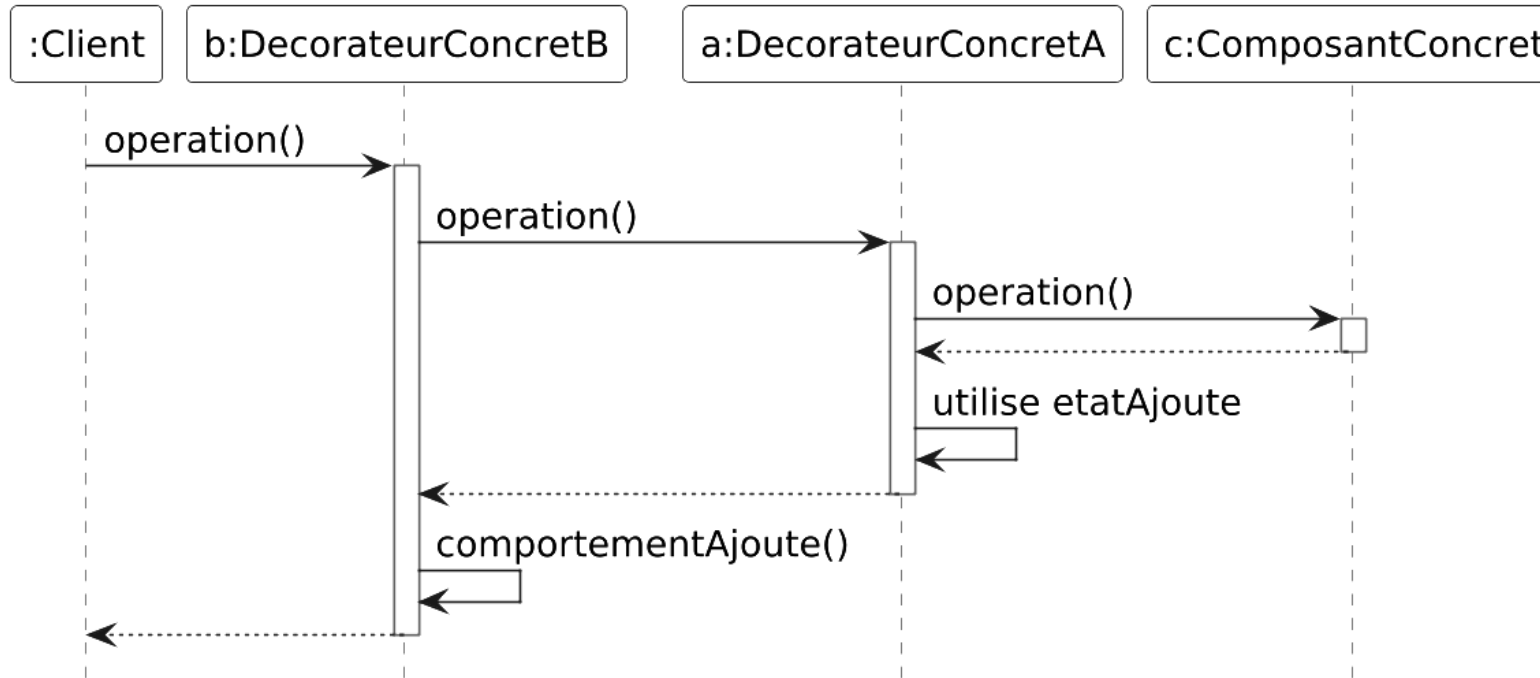
D'après Gamma, Helm, Johnson et Vlissides, 1994

Diagramme de classes



- Le Decorateur implémente Compositant et contient un Compositant. C'est ce double lien qui fait tout le pattern.
- Par défaut, il délègue simplement l'appel à l'objet qu'il enveloppe.
- Chaque DecorateurConcret ajoute un état ou un comportement avant ou après cette délégation.
- Comme un décorateur est lui-même un Compositant, on peut empiler autant de couches qu'on veut.

Diagramme de séquence



Le client ne parle qu'à la couche la plus extérieure. L'appel traverse les décorateurs de l'extérieur vers l'intérieur jusqu'au composant concret. Au retour, chaque décorateur ajoute sa contribution.

Côté code : `Composant c = new DecorateurConcretB(new DecorateurConcretA(new ComposantConcret()));`

Qui fait quoi ?

Classe participante	Description
Composant (Component)	Interface commune aux objets de base et aux décorateurs. C'est le seul type que le client manipule.
ComposantConcret (ConcreteComponent)	L'objet de départ, celui qu'on veut enrichir. Il ne sait pas qu'il peut être décoré.
Decorateur (Decorator)	Classe abstraite qui implémente Composant et garde une référence vers un Composant. Par défaut, elle délègue.
DecorateurConcret (ConcreteDecorator)	Ajoute un état ou un comportement avant ou après la délégation. On en écrit un par option.
Client	Construit la pile de décorateurs puis utilise le résultat comme un simple Composant.

S et O : une classe, un rôle, et on étend sans modifier

S Responsabilité unique

Cheddar ne sait faire qu'une chose : ajouter du cheddar et 1 € au prix. TacosPoulet ne connaît que la recette de base.

Si le prix du bacon change, une seule classe bouge : Bacon. Dans la version à booléens, il fallait toucher à la classe Tacos, qui gérait tout.

O Ouvert / fermé

Le gérant veut des oignons frits ? On ajoute une classe, et Tacos, TacosPoulet et les autres suppléments ne sont pas modifiés.

```
public class OignonsFrits extends Supplement {  
    public OignonsFrits(Tacos t) { super(t); }  
    public String description() {  
        return tacos.description() + " + oignons";  
    }  
    public double prix() {  
        return tacos.prix() + 0.50;  
    }  
}
```

L, I et D : tout repose sur l'interface Composant

L

Substitution de Liskov

Partout où la borne attend un Tacos, on peut lui donner new Cheddar(new TacosPoulet()) sans qu'elle voie la différence. À condition que le décorateur respecte le contrat : un supplément qui rendrait un prix négatif casserait la substitution.

I

Ségrégation des interfaces

L'interface Tacos n'a que deux méthodes. Chaque décorateur doit toutes les implémenter, ne serait-ce que pour déléguer. Plus l'interface est petite, plus le pattern reste léger.

D

Inversion des dépendances

Supplement dépend de l'abstraction Tacos, jamais de TacosPoulet. La borne aussi ne connaît que Tacos. On peut donc décorer n'importe quel tacos, même une recette qui n'existe pas encore.

Exemple : ajouter les oignons frits

	Version à booléens	Version Décorateur
Classes modifiées	Tacos (attribut, setter, prix(), description())	Aucune
Classes ajoutées	Aucune	OignonsFrits (une dizaine de lignes)
À retester	Tacos et toutes ses sous-classes	OignonsFrits seul
Risque de régression	Réel : on touche au cœur du calcul	Quasi nul : le code existant ne bouge pas

Composer plutôt qu'hériter. C'est un des conseils de base du GoF, et le Décorateur en est l'application directe. En bonus, chaque supplément se teste isolément, en l'appliquant à un tacos factice.

Ce qu'il faut savoir avant de l'utiliser

L'ordre compte

Une réduction de 30 % appliquée avant ou après un supplément ne donne pas le même prix.

Beaucoup de petites classes

Elles se ressemblent toutes : pour quelqu'un qui découvre le projet, c'est vite déroutant.

L'identité se perd

Un Cheddar n'est pas un TacosPoulet : `instanceof` renvoie `false`, et comparer avec `equals` n'a plus vraiment de sens.

Retirer une couche

Enlever le cheddar au milieu de la pile n'est pas prévu : il faut reconstruire l'objet.

Construction verbeuse

`new Gratine(new Cheddar(new TacosPoulet()))` devient lourd. On l'accompagne souvent d'une fabrique ou d'un builder.

Grosses interfaces

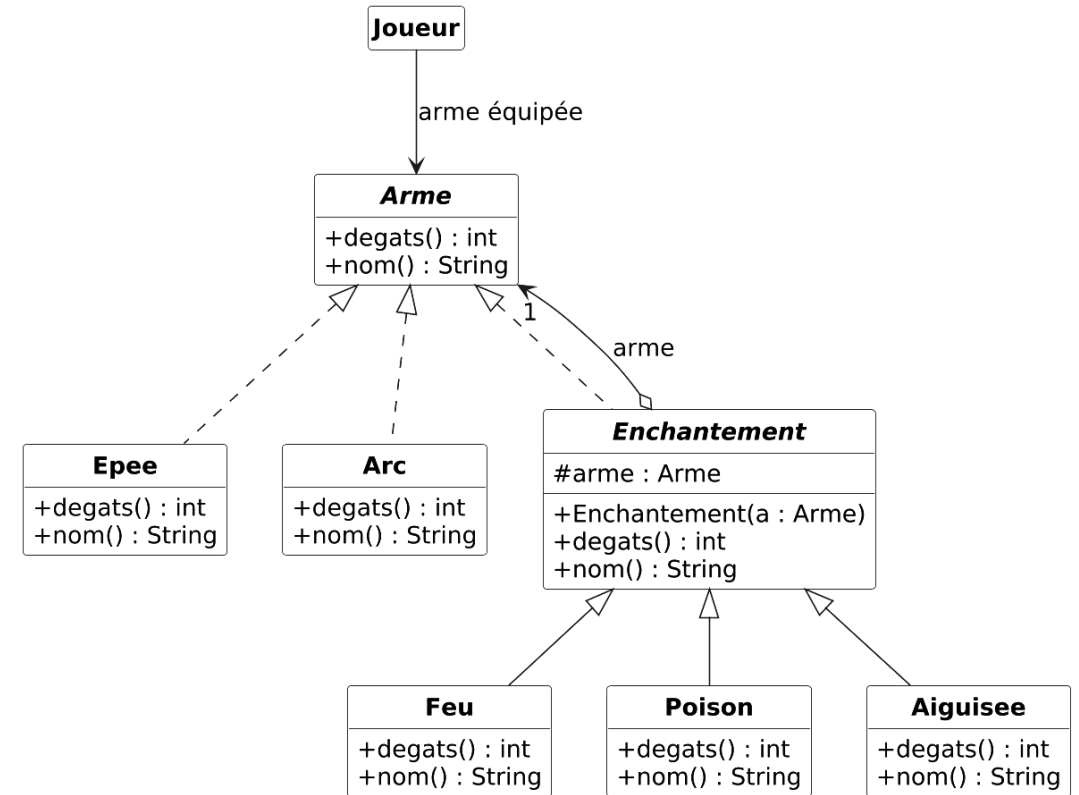
Si Composant a vingt méthodes, chaque décorateur doit toutes les déléguer. Et le débogage traverse de nombreuses couches.

Des enchantements sur une arme

Dans un RPG, le joueur ramasse des enchantements pendant la partie et les applique à son arme :

- Feu : +5 dégâts
- Poison : +3 dégâts
- Affûtage : dégâts $\times$ 2
- Ils se cumulent, dans l'ordre où le joueur les applique

Tout se décide pendant la partie : c'est typiquement un besoin de composition dynamique, donc un cas pour le Décorateur.



Le code

```

public abstract class Enchantement implements Arme {
    protected final Arme arme;
    protected Enchantement(Arme arme) { this.arme = arme; }
}

public class Feu extends Enchantement {
    public Feu(Arme arme) { super(arme); }
    public int degats() { return arme.degats() + 5; }
    public String nom() { return arme.nom() + " de feu"; }
}

public class Aiguisee extends Enchantement {
    public Aiguisee(Arme arme) { super(arme); }
    public int degats() { return arme.degats() * 2; }
    public String nom() { return arme.nom() + " aiguisée"; }
}

```

```

Arme arme = new Epee();
arme = new Feu(arme);
arme = new Aiguisee(arme);
arme.degats();
// (12 + 5) * 2 = 34

```

```

Arme autre =
    new Feu(new Aiguisee(new Epee()));
autre.degats();
// 12 * 2 + 5 = 29

```

Mêmes enchantements, ordre différent, résultat différent. C'est la limite vue juste avant, mais dans un jeu c'est souvent voulu.

Comparaison avec le diagramme générique

Enchantements d'armes

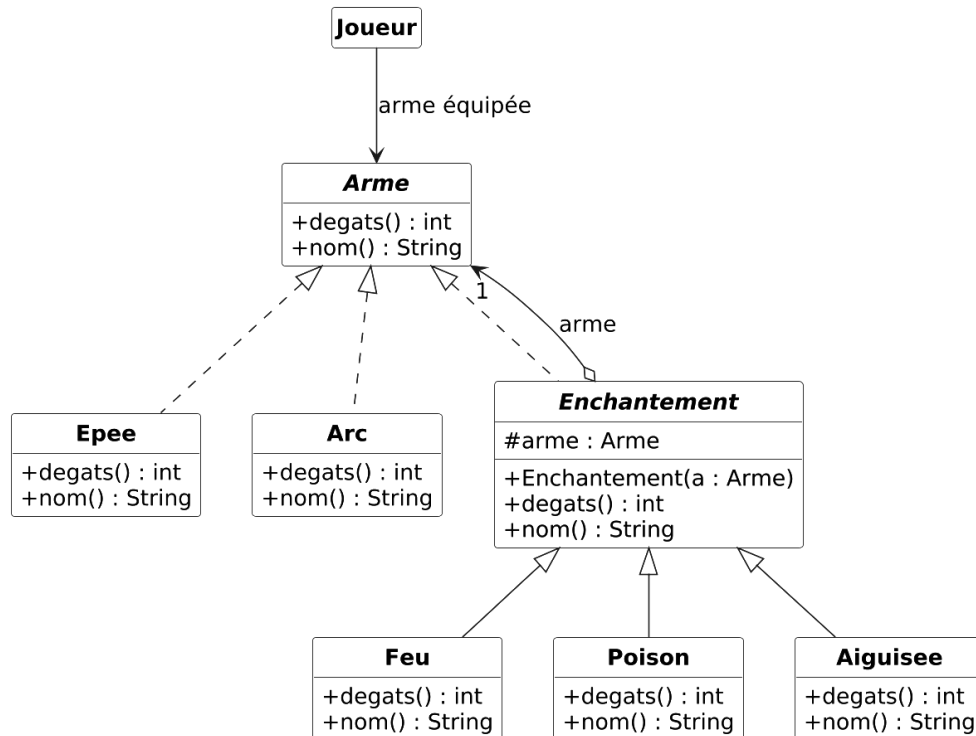
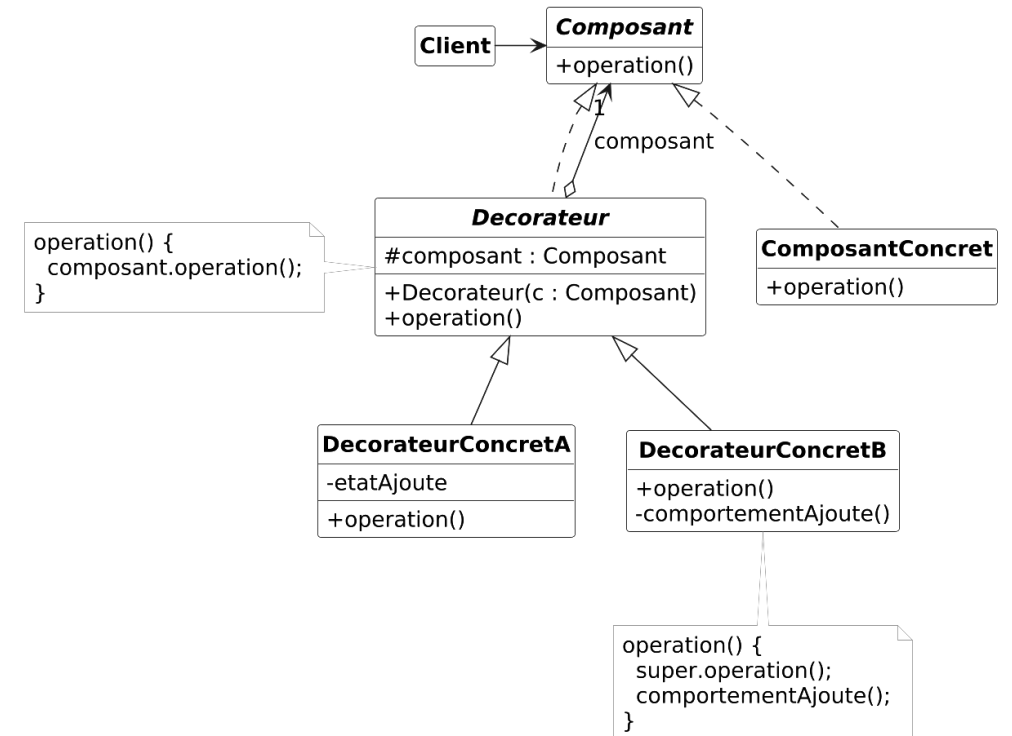


Diagramme générique



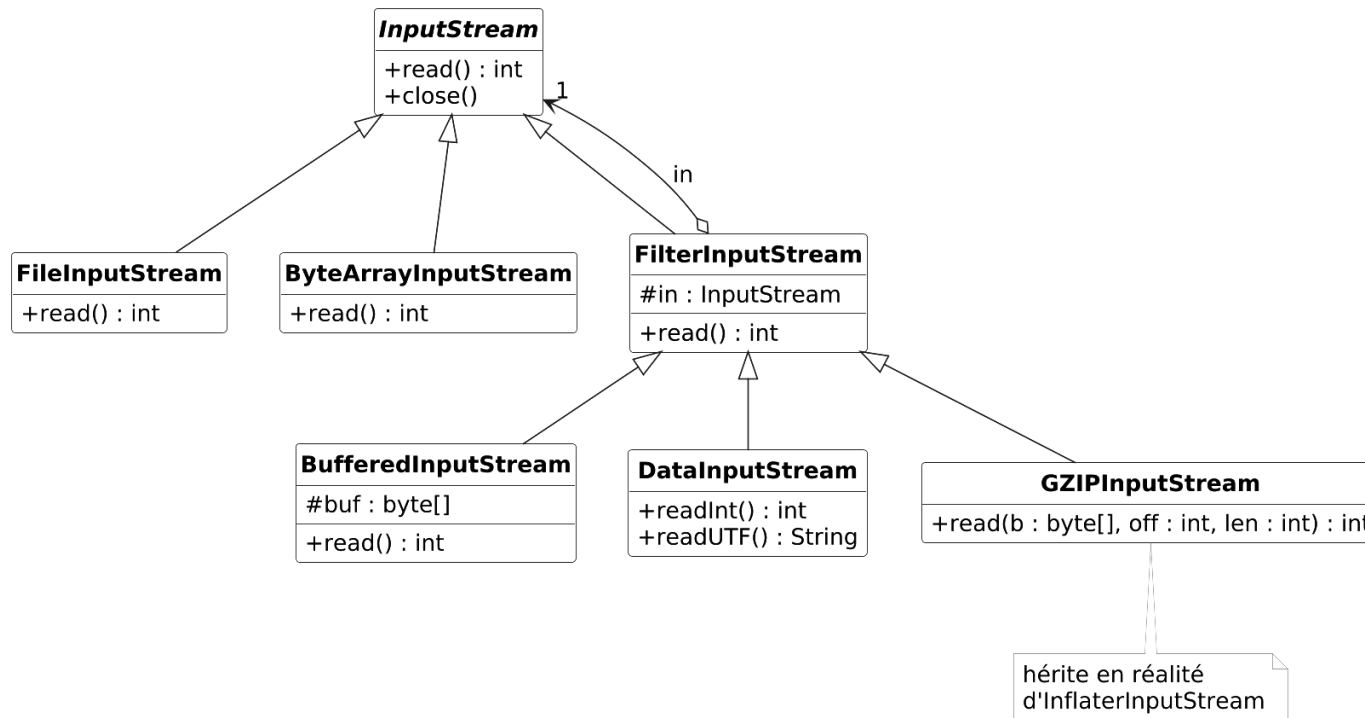
Arme = Composant

Epee, Arc =
ComposantConcret

Enchantement = Decorateur

Feu, Poison, Aiguisee =
DecorateurConcret

Les flux de java.io



- FileInputStream lit des octets bruts dans un fichier.
- GZIPInputStream décompresse au passage.
- BufferedInputStream lit par blocs au lieu d'aller sur le disque à chaque octet.
- DataInputStream ajoute readInt(), readUTF()...
- Même principe avec new BufferedReader(new FileReader("f.txt")), qu'on écrit souvent en TP.

```

DataInputStream in = new DataInputStream(
    new BufferedInputStream(
        new GZIPInputStream(
            new FileInputStream("scores.gz"))));
    
```

```

int meilleurScore = in.readInt();
    
```

Comparaison avec le diagramme générique

java.io (JDK)

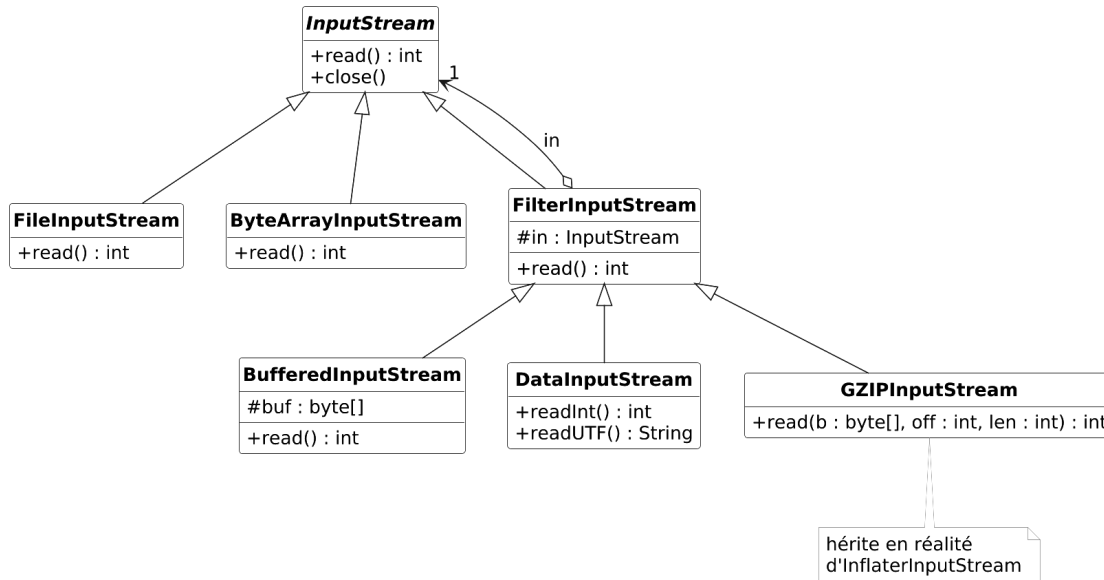
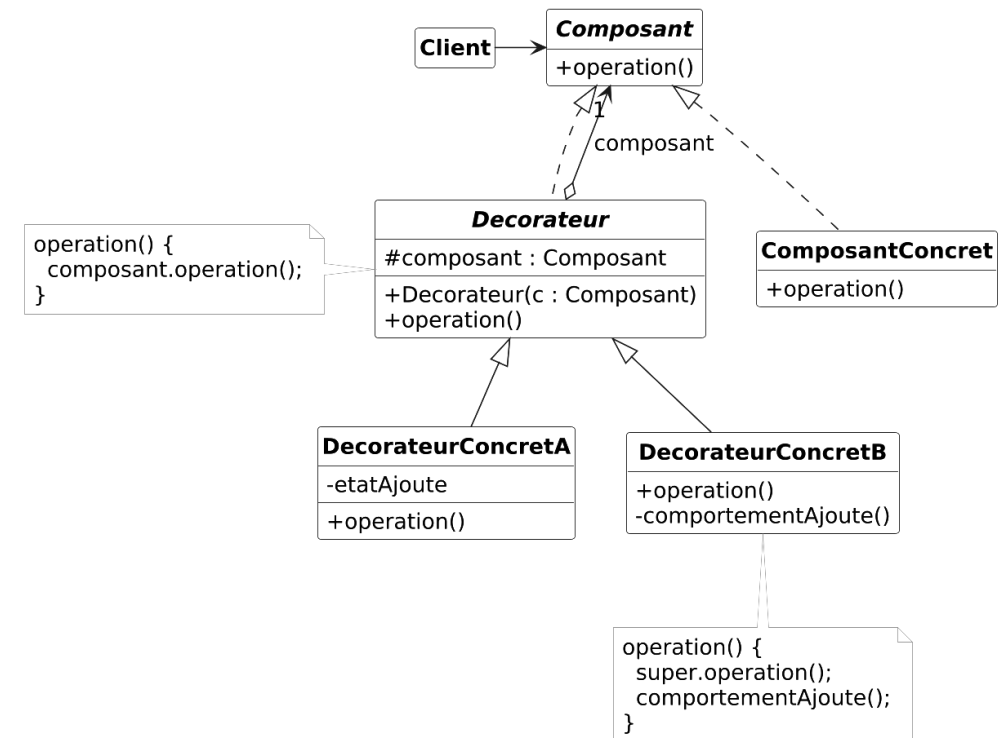


Diagramme générique



InputStream = Composant

FileInputStream =
ComposantConcret

FilterInputStream =
Decorateur

Buffered, Data, GZIP... =
DecorateurConcret

Le Décorateur ailleurs, et un faux ami

Dans le JDK

```
List<String> lecture =  
    Collections.unmodifiableList(noms);  
List<String> partagee =  
    Collections.synchronizedList(noms);
```

Ces méthodes renvoient la liste d'origine emballée : même interface List, avec un comportement en plus (refuser les modifications, synchroniser les accès).

Attention : @decorator en Python

```
@chronometre  
def calculer_moyenne(notes):  
    ...
```

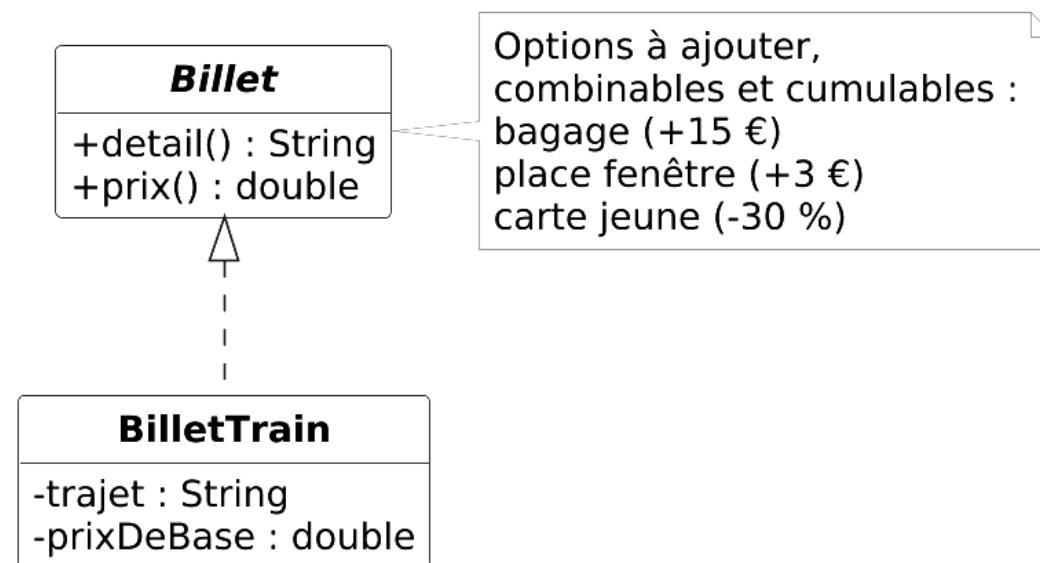
Même idée d'emballage, mais ce n'est pas le pattern du GoF : on enveloppe une fonction au moment de sa définition, pas un objet qui partage une interface avec ses décorateurs. Le mot est le même, le mécanisme est différent.

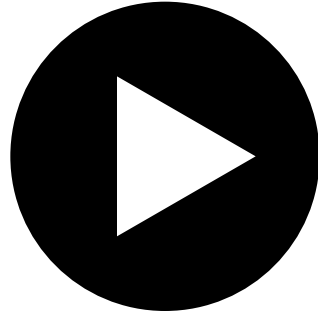
Avant la vidéo : la billetterie de train

Une appli vend des billets Limoges > Paris. Un billet a un prix de base, et le voyageur ajoute des options :

- bagage supplémentaire : +15 €
- place côté fenêtre : +3 €
- carte jeune : -30 % sur le total

Ce qu'on va coder : l'interface `Billet`, le décorateur `OptionBillet`, les trois options, puis un main qui montre que l'ordre des options change le prix.



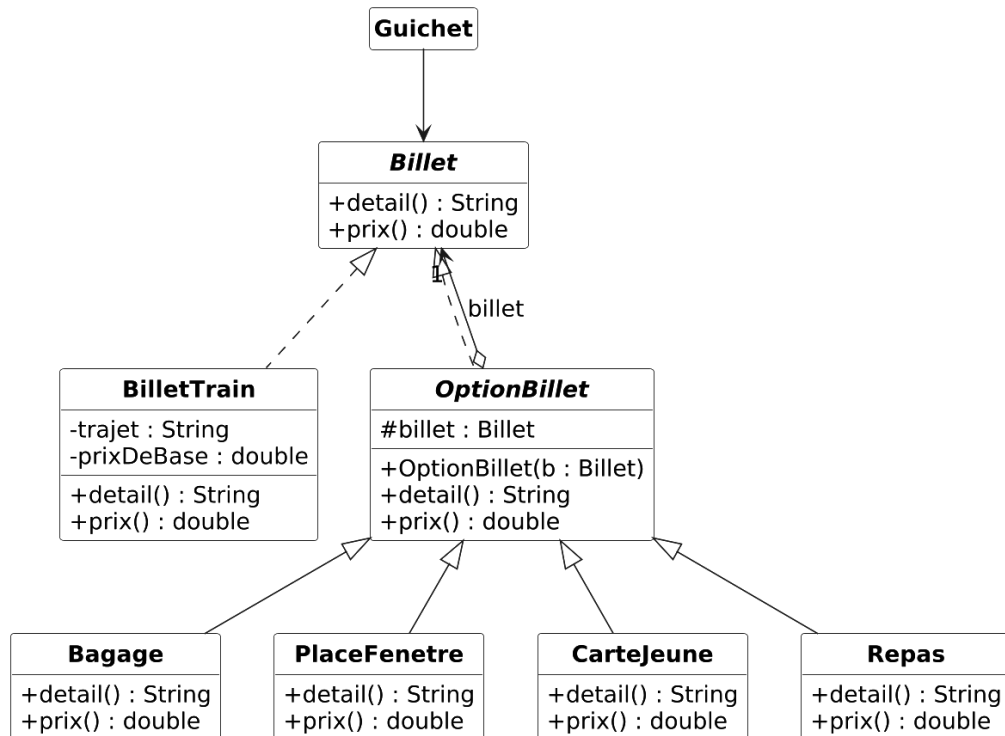


La billetterie de train

<https://www.youtube.com/watch?v=r2jp8MIHZd8>

Notre code correspond bien au pattern

Rétro-conception du live coding



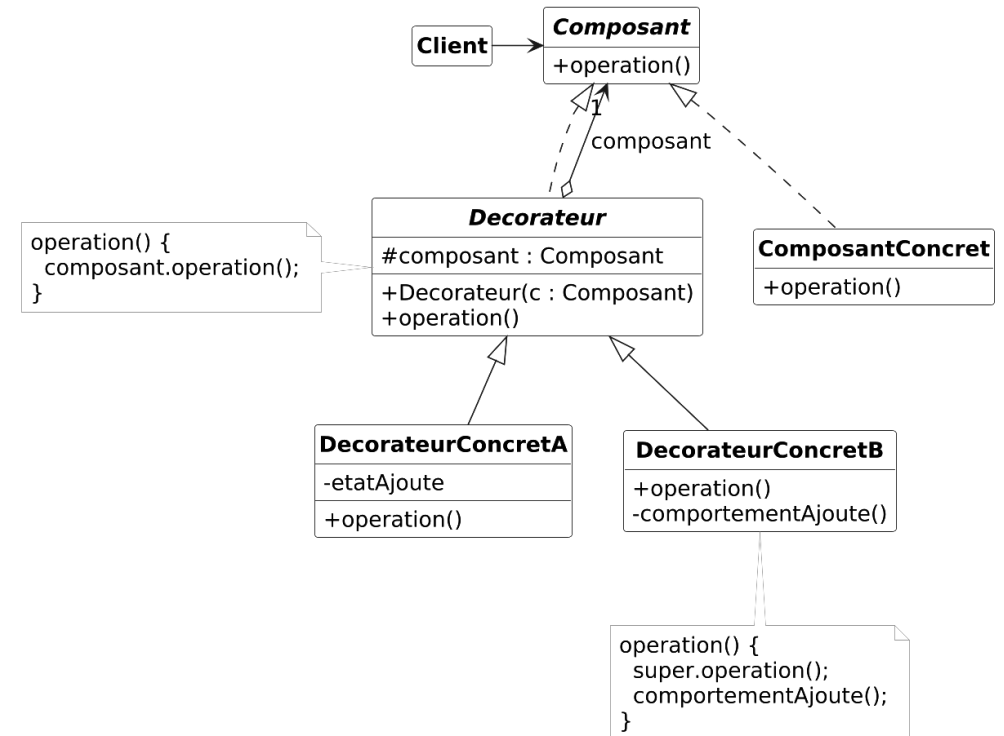
Billet = Composant

BilletTrain =
ComposantConcret

OptionBillet = Decorateur

Bagage, PlaceFenetre,
CarteJeune, Repas =
DecorateurConcret

Diagramme générique



À retenir

1. Un décorateur a la même interface que l'objet qu'il enveloppe, et il le contient.
2. On ajoute des comportements à l'exécution, en empilant des couches, sans toucher aux classes existantes.
3. Il évite l'explosion de sous-classes et applique directement le principe ouvert/fermé.
4. Attention à l'ordre des couches, au nombre de petites classes et à la perte d'identité de l'objet.

On passe au Kahoot !

Bibliographie et webographie

Livres

Gamma E., Helm R., Johnson R., Vlissides J., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

Freeman E., Freeman E., Sierra K., Bates B., Head First Design Patterns, O'Reilly, 2004, chapitre 3.

Martin R. C., Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2002.

Martin R. C., Clean Architecture, Prentice Hall, 2017, partie III.

Meyer B., Object-Oriented Software Construction, Prentice Hall, 1988.

Alexander C. et al., A Pattern Language, Oxford University Press, 1977.

Liskov B., Wing J., « A Behavioral Notion of Subtyping », ACM TOPLAS, vol. 16, n° 6, 1994.

Sites

Refactoring.Guru, Décorateur : <https://refactoring.guru/fr/design-patterns/decorator>

SourceMaking, Decorator Design Pattern : https://sourcemaking.com/design_patterns/decorator

Java Design Patterns, Decorator : <https://java-design-patterns.com/patterns/decorator/>

Oracle, Java SE 21 API, FilterInputStream : <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/FilterInputStream.html>

Python Software Foundation, glossaire, decorator : <https://docs.python.org/3/glossary.html#term-decorator>

Wikipédia, Décorateur (patron de conception) : [https://fr.wikipedia.org/wiki/Décorateur_\(patron_de_conception\)](https://fr.wikipedia.org/wiki/Décorateur_(patron_de_conception))

IAGraphie

Outil utilisé	Nature de l'apport	Prompt de synthèse
Claude (Anthropic)	Mise en forme du diaporama : palette, typographie, mise en page, fond blanc pour le code et les diagrammes.	« Propose une mise en page claire et cohérente pour notre diaporama sur le Décorateur, avec le code et l'UML sur fond blanc. »
Claude (Anthropic)	Synthèse des notions clés issues des ouvrages de référence (GoF, Head First Design Patterns, R. C. Martin). Sources vérifiées par le groupe.	« Résume ce que disent le GoF, Head First Design Patterns et R. C. Martin sur le Décorateur et SOLID, avec les références exactes. »
Claude (Anthropic)	Premières propositions des exemples de code et du QCM. Relus, testés et retravaillés par le groupe.	« À partir des consignes R3.04, propose des exemples de code et des questions type QCM sur le Décorateur. »

Le live coding a été réalisé sans aucun assistant de code activé dans l'IDE.

Merci !

Des questions ?